

LoreBot

Last updated: 2026-05-14

LoreBot is a document-reading AI assistant built into ArtHound. Given an asset or a vendor dispatch, it reads the attachments from Supabase Storage, LoreBot is a document-reading AI assistant built into ArtHound. Given an asset or a vendor dispatch, it reads the attachments from Supabase Storage, PDFs, text files, images, and answers questions about their content using Claude Haiku.

This is a proof-of-concept feature. It is clearly marked as such in the UI and backend. Do not use it with confidential or sensitive data.

What It Does

LoreBot allows studios and vendors to have a conversation about the files attached to a production asset. Useful for: - Summarising a long brief document - Quick-referencing details from a spec sheet - Cross-referencing information across multiple attached files - Answering questions about concept art direction from image files

It reads attachment content directly from Supabase Storage (the content-addressed layer used by the rest of the attachment pipeline). It never calls out to the original source tool during a chat session, it works from the stored copy.

Prerequisites

Attachments must have been copied to Supabase Storage before LoreBot can read them. LoreBot includes a **replication step**, if uncopied attachments are detected, the chat endpoint returns `{needs_replication: true, uncopied: [ ... ]}` instead of an answer. The frontend must call `POST /api/lorebot/replicate` first, then retry the chat.

Access

Available to both studio and vendor users.

- **Studio users:** can chat about any asset in their own org's `replicated_assets`
- **Vendor users:** can chat about dispatches received from studios (non-revoked only)

The correct context (asset vs. dispatch) is specified in the request body via `asset_id` or `dispatch_id`.

Supported Content Types

Type	Handling
Images (JPEG, PNG, GIF, WebP)	Passed to Claude Haiku as base64-encoded <code>image</code> blocks. Up to 4 images per chat turn.
PDF	Text extracted via <code>pypdf</code> ; up to 40,000 characters per file. Scanned PDFs (no text layer) return a note explaining extraction failed.
Plain text, Markdown, CSV, JSON, XML	Decoded as UTF-8; up to 40,000 characters per file.
All other types	Listed in the system prompt as unsupported; LoreBot notes they couldn't be read.

If a file fails to load from Storage, it is listed in the system prompt as a read error and skipped.

Chat Behaviour

System prompt

Each request builds a system prompt with: - LoreBot's identity and PoC disclaimer - A list of all attachments (filenames, type classification, char count or image status) - Extracted text content from all supported text/PDF files - Rules: answer only from provided content, never fabricate, flag truncated content

Images are injected as content blocks into the last user message (Claude's native vision API).

Prompt caching (`cache_control: {type: "ephemeral"}`) is applied to the system prompt to reduce latency on follow-up questions about the same attachment set.

Scope rules

LoreBot is instructed to: - Base answers only on the content of the provided files - Clearly state when something is not in the files - Never fabricate details not present in the content - Note when content was truncated (40k character limit)

Replication

`POST /api/lorebot/replicate` triggers synchronous attachment copying for an asset or dispatch, the same underlying pipeline as the rest of the attachment system, but called on-demand rather than on-dispatch or on-first-view.

For studio assets, `_replicate_studio_asset()` walks the asset's `meta` dict, identifies uncopied attachment items (those without a `content_hash`), downloads each from the source tool (Airtable/Jira), hashes and uploads to Supabase Storage, then patches `content_hash` back into

`replicated_assets.meta`. Expired Airtable URLs are refreshed by fetching the single record directly before re-attempting the download.

For vendor dispatches, the existing `copy_payload_attachments(dispatch_id)` from `lib/attachments.py` is called.

API Reference

All endpoints are under `/api/lorebot`. JWT required.

Method	Path	Auth	Description
GET	<code>/items</code>	studio or vendor	List assets/dispatches that have at least one attachment
POST	<code>/replicate</code>	studio or vendor	Copy uncopied attachments to Storage (synchronous)
POST	<code>/chat</code>	studio or vendor	Chat about attachment content

GET `/api/lorebot/items`

Returns a list of items with attachments for the calling user: - Studio: up to 500 assets from `replicated_assets` that have any attachment-type meta field - Vendor: up to 100 non-revoked dispatches that have any attachment-type field in `payload_data`

```
[
  { "id": "canonical-uuid-or-dispatch-id", "label": "Hero Character", "type": "asset" },
  { "id": "dispatch-uuid", "label": "Studio A - 2026-05-10", "type": "dispatch" }
]
```

POST `/api/lorebot/replicate`

```
{ "asset_id": "canonical-uuid" }
// or
{ "dispatch_id": "dispatch-uuid" }
```

Returns `{"status": "done"}` on success. Runs synchronously, may be slow for assets with many large attachments.

POST `/api/lorebot/chat`

```
{
  "messages": [
    { "role": "user", "content": "What's the brief for this character?" },
    ...
  ],
  "asset_id": "canonical-uuid"
}
```

Returns one of:

Needs replication:

```
{
  "needs_replication": true,
  "uncopied": [
    {"filename": "brief.pdf", "field_key": "Brief", "idx": 0}
  ]
}
```

Answer:

```
{ "answer": "The brief describes a warrior character with..." }
```

No attachments:

```
{ "answer": "This asset has no attachments for me to read." }
```

Known Gaps

PoC status: LoreBot is explicitly a proof of concept. It has not been through a production security review. Do not use with confidential pre-release IP.

No multi-turn context beyond what the client sends: like NumberBot, LoreBot has no persistent memory. The client must send the full message history each turn.

Image limit: only the first 4 images are passed to the model. Additional images are listed in the system prompt but their visual content is not available to the model.

PDF text extraction quality: `pypdf` works well for digitally-produced PDFs but fails on scanned/image-based PDFs. OCR is not available.

No streaming: responses are returned as a single JSON payload, not streamed. Long answers from complex documents may have noticeable latency.

Replication is synchronous and slow: `POST /replicate` blocks until all attachments are copied. For assets with many large files, this can time out at the HTTP layer before completing.